

Design Justification Report

Edge CNN Accelerator for Industrial AI Applications

ECE 410/510 — Hardware for Artificial Intelligence and Machine Learning

Spring 2026 | Naveen Babu Vanamala

Section 1: Problem and Motivation

Industrial machine vision systems — defect inspection on assembly lines, equipment fault detection from thermal cameras, and object recognition on embedded vision sensors — require real-time CNN inference at the network edge. Deploying inference on a general-purpose CPU is untenable: the host CPU consumes 28 W TDP, cannot guarantee sub-millisecond latency, and wastes most of its execution time on memory transactions rather than computation.

The target kernel for this accelerator is a 3×3 INT8 convolutional layer, which accounts for the dominant fraction of inference time in shallow industrial CNNs. M1 profiling (`project/m1/sw_baseline.md`) measured the Python `im2col` software baseline on an Intel Core i7-1165G7 (2.80 GHz, LPDDR4x 51.2 GB/s) running a CNN with convolutional layers, ReLU activations, max-pooling, and fully connected layers on synthetic industrial defect data. The `Conv2D_im2col` kernel accounted for **80.6% of total runtime** (31.2 of 38.7 seconds over three training epochs). Re-measured on a 32×32 single-image inference workload for M4 comparison, the pure Python `im2col` loop achieves **5.91 MFLOPs/s** and takes **10.966 ms per image** (see `project/m4/bench/benchmark_data.csv`, `rows` `sw_baseline_throughput` and `sw_baseline_time_per_image`).

Custom hardware addresses both the throughput and power problems simultaneously. A weight-stationary parallel PE array avoids repeated traffic for weights by loading each weight once and broadcasting the same pixel data to all processing elements in parallel. The target platform is the sky130A open-source process design kit (PDK) with the sky130_fd_sc_hd standard-cell library, which is representative of a low-power edge ASIC process.

The specific motivation for hardware acceleration is threefold: (1) the 2.48 FLOPs/byte arithmetic intensity of the convolutional kernel places it in the memory-bound region on a CPU, where the only path to higher throughput is reducing memory transactions per FLOP — exactly what weight-stationary on-chip buffering achieves; (2) the i7's 28 W TDP is two to three orders of magnitude above the power envelope of an edge IoT sensor node; and (3) the serial per-pixel Python loop cannot exploit the data-parallel structure of the convolution operation.

Section 2: Roofline Analysis

The roofline analysis establishes the theoretical performance ceiling for the 3×3 INT8 convolutional kernel on the target platform and locates the measured M4 accelerator on that ceiling.

Arithmetic intensity. For a single 3×3 patch with 4 output channels and 1 input channel: 9 multiply-accumulate operations × 4 PEs × 2 FLOPs/MAC = 72 FLOPs. The input data transferred per patch is 9 INT8 pixel bytes + 4 INT8 weight bytes (loaded once; amortized) + 16 bytes output = 29 bytes minimum. Lower-bound arithmetic intensity: $72 / 29 \approx \mathbf{2.48 \text{ FLOPs/byte}}$ (consistent with M1 measurement of 2.51 FLOPs/byte; the difference is rounding in the byte-count model). All numbers are in `project/m4/bench/benchmark_data.csv` (row `arithmetic_intensity`).

Sky130A roofline ceilings. The `sky130_fd_sc_hd` library running at 100 MHz supports INT8 MAC operations at one result per PE per clock. With 4 PEs each accumulating one 9-tap dot-product in 9 cycles, the ideal compute rate is $4 \times 2 \text{ FLOPs/MAC} \times 100 \text{ MHz} = \mathbf{800 \text{ MFLOPs/s}}$ peak compute ceiling. The CF09 roofline analysis (`project/codefest/cf09/benchmarks/roofline_analysis.md`) measured the sky130A bandwidth ceiling at **330 MFLOPs/s** at AI = 2.48 and the compute ceiling at **800 MFLOPs/s**, placing the ridge point at approximately 6.0 FLOPs/byte.

Measured M4 position. The M4 accelerator achieves **47.7 MFLOPs/s** at AI = 2.48 FLOPs/byte (see Figure 4, `roofline_final.png`), measured directly from co-simulation timing (151 cycles/patch, `final_run.log` TIMING SUMMARY). This is **6.9× below the bandwidth ceiling** and **16.8× below the compute ceiling**. The design operates in neither the bandwidth-bound nor compute-bound regimes — it is **AXI-overhead-bound**: 142 of 151 cycles per patch are spent on AXI4-Lite bus handshakes, and only 9 cycles perform MAC accumulation (5.96% MAC utilization, measured). Closing the gap to the bandwidth ceiling requires replacing the per-pixel AXI write sequence with a streaming interface. This is discussed in Section 9.

How the roofline shaped the architecture. The roofline analysis directly motivated the weight-stationary dataflow: reusing each weight across all 9 pixel positions reduces weight-reload bandwidth from $4 \times 9 = 36$ to 4 byte fetches per patch invocation, increasing the effective arithmetic intensity. It also motivated the choice of 4 parallel PEs: doubling from 2 to 4 PEs doubles compute throughput at near-zero bandwidth cost because the same pixel is broadcast to all PEs from a single register.

Section 3: Precision and Data Format

Format: INT8 symmetric quantization (Q7.0 representation).

Activations and weights are both represented as 8-bit signed integers in the range $[-128, +127]$. The scale factor applied to weights is $S = \max|W| / 127$, derived from the M2 precision analysis (`project/m2/precision.md`). Pixel activations are mapped from the $[0, 255]$ range via the same

scale factor. The accumulator is **INT32 (32-bit signed)**, which prevents overflow for any combination of 9 INT8 inputs and 9 INT8 weights: the worst-case accumulated value is $9 \times 127 \times 127 = 145,161$, which is well within the INT32 range of $\pm 2,147,483,647$. No saturation logic is needed.

Why INT8 over alternatives.

- vs. *FP16*: A floating-point multiply unit consumes 5–8× more silicon area and power than an equivalent INT8 MAC unit. At the sky130A process node targeting an edge power envelope below 10 mW, FP16 is infeasible. INT8 also avoids IEEE 754 rounding complexity in hardware.
- vs. *INT4*: INT4 (range –8 to +7) introduces quantization error unacceptable for industrial defect detection. Published benchmarks on ResNet-50 show 2–5% top-1 accuracy loss at INT4 versus less than 0.5% at INT8. The application-level specification (miss rate below 1%) cannot tolerate 2–5% accuracy degradation.
- vs. *BF16*: BF16 preserves the FP32 exponent range and is optimized for training, not edge inference. It requires a wider datapath with no accuracy benefit for quantized inference workloads.

Error analysis (M2, 100 samples). INT8 dot-products were compared against FP32 ground truth on 100 randomly sampled 3×3 convolutional patches from synthetic industrial defect images. Results: mean absolute error = 0.0031, maximum absolute error = 0.0089 (on a normalized scale [–1, 1]), standard deviation = 0.0018, accuracy delta < 0.3%. The maximum error of 0.0089 is below 1 LSB of the INT8 representation ($1/127 \approx 0.0079$), confirming that error is bounded within the expected uniform quantization model. All twelve co-simulation result checks in `project/m4/sim/final_run.log` produce bitwise-exact results against independently computed Python reference values, confirming zero hardware accumulation error.

Section 4: Dataflow and Architecture

Dataflow pattern: weight-stationary.

In a weight-stationary dataflow, each processing element (PE) holds one weight value fixed for the duration of a patch computation. The same pixel value is broadcast from a single input register to all PEs in parallel on each clock cycle. This eliminates weight-reload bandwidth during computation: the 4 weights are loaded once via the AXI4-Lite WEIGHT_IN register (four sequential writes), then held in on-chip flip-flops (`weight_mem[0:3]` in `compute_core.sv`) while all 9 pixel taps stream through. Weight-stationary is the correct choice for a 3×3 kernel with few output channels: the weight array is small (4×1 byte = 4 bytes), making on-chip weight storage cheap, and the pixel broadcast is efficient at 4 PEs.

Note on "systolic" terminology. This design is a weight-stationary parallel broadcast array, not a systolic array. In a true systolic array, data propagates between PEs with regular inter-PE dataflow; here all PEs receive the identical pixel from a single register with no inter-PE communication. The architecture is correctly described as a weight-stationary parallel PE array. References to "systolic" in earlier milestones were imprecise and do not apply to this design.

Compute engine. The compute engine (`project/m4/rtl/compute_core.sv`, `conv_pe.sv`) consists of four `conv_pe` instances in parallel. Each `conv_pe` holds one INT8 weight, multiplies it against the incoming INT8 pixel every clock cycle when `valid_in` is asserted, and accumulates into a 32-bit signed register. After 9 taps (`KERNEL_SIZE = 9`), the PE asserts `valid_out` and presents the completed dot-product on `accum_out`. The tap counter (`tap_count[3:0]`) resets to zero after the 9th tap and the accumulator clears to zero for the next patch.

What one PE computes. Each PE computes $\text{result} = \text{weight}_i \times \text{sum}(\text{pixels}[0..8])$ — i.e., a single weight applied to all 9 spatial positions of one 3×3 input patch. This is the weight-stationary inner loop of a depth-wise or point-wise convolution style accumulation. Four PEs in parallel compute four such weighted sums simultaneously, one per output channel. See Figure 1 (block diagram) and Figure 2 (dataflow diagram).

Memory hierarchy.

- *Weight storage:* 4 × 8-bit flip-flop registers in `compute_core.sv` (`weight_mem`). Loaded via AXI4-Lite WEIGHT_IN once per kernel application.
- *Pixel staging:* One 8-bit register (`reg_pixel`) in `axi4lite_slave`. The host writes PIXEL_IN and CTRL in sequence for each of the 9 taps.
- *Output buffering:* Four 32-bit result registers (`reg_result[0:3]`) in `top.sv`. Written when `core_result_valid` is asserted; read back via AXI4-Lite RESULT addresses (0x10–0x1C). A sticky `reg_done` flag at 0x20 signals completion; cleared automatically when read.

Data path. The host (AXI4-Lite master) writes to `cnn_interface`, which contains `axi4lite_slave`. The slave decodes the address and drives `pixel_out`, `weight_out`, and `valid_out`. In `top.sv`, the write-channel FSM captures weight writes with a one-cycle pipeline delay and drives `weight_wr`, `weight_addr`, and `weight_din` into `compute_core`. A one-cycle `valid_in_delayed` register compensates for the pixel register settling time. The `compute_core` instantiates four `conv_pe` units in a `generate` loop and packs their outputs onto a 128-bit result bus. On `core_result_valid`, `top.sv` captures all four results into `reg_result[0:3]` and sets `reg_done`.

Section 5: Hardware Interface

Interface implemented: AXI4-Lite (AMBA AXI4-Lite Protocol Specification).

AXI4-Lite was selected over full AXI4 for three reasons. First, the data transfer volume per patch is small: 4 weight bytes + 9 pixel bytes = 13 bytes, which does not justify burst machinery. Second, AXI4-Lite has a straightforward synthesizable implementation with no FIFO or burst counter logic, reducing design risk. Third, AXI4-Lite is universally supported by FPGA and ASIC platform interconnects, making the accelerator compatible with standard SoC design flows.

Register map.

Address	Register	Width	Direction	Description
0x00	PIXEL_IN	[7:0]	W/R	INT8 pixel value for current tap
0x04	WEIGHT_IN	[7:0]	W/R	INT8 weight value to load into PE
0x08	CTRL	[0]	W/R	Write 1: trigger valid_in strobe
0x0C	STATUS	[0]	R	Set when computation complete
0x10	RESULT0	[31:0]	R	PE0 32-bit dot-product result
0x14	RESULT1	[31:0]	R	PE1 32-bit dot-product result
0x18	RESULT2	[31:0]	R	PE2 32-bit dot-product result
0x1C	RESULT3	[31:0]	R	PE3 32-bit dot-product result
0x20	DONE	[0]	R	Sticky flag; auto-cleared on read

Measured effective bandwidth. From co-simulation (`final_run.log`): each pixel tap requires 2 AXI transactions (PIXEL_IN write + CTRL write). The log shows consecutive tap timestamps separated by 140 ns = 14 clock cycles, so each AXI write transaction takes approximately 7 cycles. At 100 MHz, effective write bandwidth = 1 byte / 70 ns $\approx$ **14.3 MB/s** per transaction slot. Over 9 taps (18 transactions) plus 5 overhead reads: 23 transactions $\times$ $\sim$ 7 cycles = $\sim$ 161 cycles — slightly higher than the measured 151 because the DONE and result reads resolve faster. The measured 14 cycles per tap (both writes together) is directly readable from the log timestamps.

Interface-bound diagnosis. The measured design devotes **142 of 151 cycles per patch (94.0%) to AXI transactions** (row `hw_axi_utilization` in `benchmark_data.csv`). The design is AXI-overhead-bound, not bandwidth-bound in the classical sense: the bottleneck is transaction count and handshake latency, not raw bus throughput. A streaming FIFO would address this by reducing per-patch transaction count from 18 to 1 (a single burst or FIFO-fill). See Section 9.

Section 6: Verification

The design was verified through three levels of testbenches across M2 and M3, all targeting the same RTL committed in `project/m4/`.

M2 unit-level verification (`project/m2/tb/`).

- `tb_compute_core.sv`: Directly drives `compute_core` with sequences of `valid_in` pulses and specific pixel/weight pairs. Verified: accumulation over 9 taps, weight loading via `weight_wr/weight_addr`, `result_valid` pulsing, reset behavior. Reference values computed in Python and compared bitwise.
- `tb_interface.sv`: Exercises the AXI4-Lite write and read channels in isolation. Verified: write FSM (IDLE → DATA → RESP), read FSM (IDLE → DATA), register decode for all addresses, `pixel_out/weight_out` update.

M4 end-to-end co-simulation (`project/m4/tb/tb_top.sv`). The top-level testbench drives the complete AXI4-Lite interface of the integrated `top` module. Weights are loaded once (one-time cost), then **3 back-to-back patch invocations** are executed and timed. The test sequence per invocation is:

1. *Pixel streaming*: For each of 9 taps, write `PIXEL_IN` (0x00) then `CTRL` (0x08). Each `CTRL` write fires `valid_in` into the compute pipeline.
2. *Done poll*: Read `ADDR_DONE` (0x20) until the sticky flag is set (auto-cleared on read).
3. *Result readback*: Read `RESULT0–RESULT3` (0x10–0x1C). Compare against golden expected values.

Golden expected values. For weights [3, 7, -2, 5] and pixels [1, 2, 3, 4, 5, 6, 7, 8, 9]:

- PE0: $3 \times (1+2+\dots+9) = 3 \times 45 = \mathbf{135}$ □
- PE1: $7 \times 45 = \mathbf{315}$ □
- PE2: $-2 \times 45 = \mathbf{-90}$ □
- PE3: $5 \times 45 = \mathbf{225}$ □

The simulation log (`project/m4/sim/final_run.log`) shows all four PEs produce correct results in every one of the 3 invocations — **12 of 12 PE result checks pass** — ending with the token `PASS`. The TIMING SUMMARY in the log directly provides the measured cycle counts (151 cycles/patch) that support the benchmark numbers in Section 8. See Figure 3 (`final_waveform.png`) for the annotated waveform.

Coverage: AXI write path (address and data phase handshakes), AXI read path, weight loading and PE index incrementing, pixel register capture, `CTRL` strobe generation, tap counter advance and reset, 9-tap accumulation arithmetic, multi-invocation result consistency, `DONE` flag assertion and auto-clear behavior, 3 back-to-back patch cycles without reset.

Section 7: Synthesis Results

The design was synthesized using OpenLane 2 on the sky130A PDK (sky130_fd_sc_hd standard-cell library) with a 10 ns (100 MHz) clock constraint and AREA 0 synthesis strategy. The OpenLane 2 flow (`project/m4/synth/openlane_run.log`) ran 43 steps from Verilator lint through TritonRoute detailed routing. Timing and power numbers are from step 42 (OpenROAD STAMidPNR-3, post-global-routing STA). Area numbers are from step 13 (OpenROAD floorplan). All raw numbers are in `benchmark_data.csv`.

Area.

Metric	Value	Source
Die area	59,334 μm^2	Step 13 floorplan
Core area	51,302 μm^2	Step 13 floorplan
Standard-cell area	20,656 μm^2	Step 13 floorplan
Cell count (stdcell)	1,763 cells	Step 13 floorplan
Core utilization	40.3%	Step 13 floorplan

The dominant area contributor is the `conv_pe` array (4 instances), which accounts for approximately 52% of logic area. Each `conv_pe` contains an 8×8 signed integer multiplier (synthesized as a carry-save structure) and a 32-bit accumulator register, making the MAC datapath the largest per-cell structure. The `axi4lite_slave` (AXI FSM + register file) contributes approximately 10%, and `top.sv` glue logic contributes approximately 12%.

Timing.

Metric	Value	Corner
Setup WNS	0.000 ns	nom_tt_025C_1v80
Hold WNS	0.000 ns	nom_tt_025C_1v80
Worst setup slack	+4.062 ns	nom_tt_025C_1v80
Worst hold slack	+0.312 ns	nom_tt_025C_1v80
Setup violations	0	all corners
Hold violations	0	all corners

The design **closes timing at 100 MHz** with +4.062 ns of positive setup slack in the typical corner. No timing violations were found at any analyzed corner. The critical path runs from a weight register flip-flop through the 8×8 signed multiplier and 32-bit carry-lookahead adder to the accumulator register. The 4.062 ns of positive slack provides margin against routing parasitics from detailed routing.

Power.

Group	Internal (W)	Switching (W)	Leakage (W)	Total (W)	Fraction
Sequential	1.277×10^{-3}	1.190×10^{-5}	2.554×10^{-9}	1.289×10^{-3}	55.8%
Combinational	6.605×10^{-5}	8.318×10^{-5}	7.720×10^{-9}	1.492×10^{-4}	6.5%
Clock	4.353×10^{-4}	4.369×10^{-4}	9.138×10^{-10}	8.722×10^{-4}	37.8%
Total	1.778×10^{-3}	5.320×10^{-4}	1.12×10^{-8}	2.310×10^{-3}	100%

Total power: **2.310 mW** at 100 MHz, 1.80 V, typical corner. The dominant contributor is sequential switching (55.8%), followed by the clock tree (37.8%). This is within the target of <10 mW for edge IoT deployment.

Section 8: Benchmark Results

Measured hardware throughput. All numbers are directly traceable to `project/m4/sim/final_run.log` (TIMING SUMMARY section) and `benchmark_data.csv`.

The testbench (`tb_top.sv`) ran 3 back-to-back patch invocations after a one-time weight load. The `$time` marker at measurement start was **375 ns** and at measurement end was **4,906 ns**, giving an elapsed time of **4,531 ns = 453 clock cycles** for 3 patches. Per patch: **151 cycles = 1,510 ns = 1.51 μ s** (rows `hw_sim_cycles_per_patch` and `hw_sim_time_per_patch_ns` in `benchmark_data.csv`).

For a 32×32 image with 900 output patches (weights reloaded once at the start):

- Time per image = $900 \times 1,510 \text{ ns} = \mathbf{1.359 \text{ ms}}$
- Throughput = $64,800 \text{ FLOPs} / 1.359 \text{ ms} = \mathbf{47.7 \text{ MFLOPs/s}}$
- Samples/sec = **736 images/sec**

Speedup vs M1 software baseline.

Metric	SW Baseline (M1 loop)	HW Accelerator (M4, measured cosim)
Time per image	10.966 ms	1.359 ms
Time per patch	12.18 μ s	1.510 μs (151 cycles $\times$ 10 ns)
Throughput	5.91 MFLOPs/s	47.7 MFLOPs/s
Samples/second	91.2 img/s	736 img/s
Power	~28 W (i7-1165G7 TDP)	2.310 mW (OpenROAD, sky130A)
Energy/image	~307 mJ	~3.14 μJ (2.31 mW $\times$ 1.359 ms)

Speedup = 10.966 ms / 1.359 ms = 8.07×

Energy comparison. At 2.310 mW and 1.359 ms runtime per image, the accelerator consumes approximately **3.14 μ J per image**. The i7 baseline at 28 W TDP and 10.966 ms consumes approximately **307 mJ per image** — an energy reduction of approximately **~97,800×**. The SW energy figure uses the CPU TDP (conservative upper bound).

Roofline position (M4 measured). The M4 accelerator plots at $AI = 2.48$ FLOPs/byte, 47.7 MFLOPs/s = 0.0477 GFLOPs/s (Figure 4). This is 6.9× below the sky130A bandwidth ceiling (330 MFLOPs/s at $AI = 2.48$). The M4 measured point reflects the actual simulation result, not the M1 hypothetical design target.

Gap between measured and theoretical performance. The 151-cycle per-patch cost is dominated by AXI overhead: 9 taps $\times$ 14 cycles/tap (PIXEL_IN + CTRL each) = 126 cycles, plus 1 DONE read = 5 cycles, plus 4 result reads $\times$ 5 cycles each = 20 cycles — total 151 cycles, with only 9 cycles of actual MAC accumulation (5.96% utilization). These cycle counts are consistent with the per-tap timestamps visible in `final_run.log`.

Section 9: What Did Not Work

This section documents four specific design attempts that were not completed before the M4 deadline, one design bug discovered and fixed during the project, and lessons learned.

9.1 Streaming pixel FIFO (Task 1 — not completed).

The primary performance gap is the 94.0% AXI bus overhead per patch. The planned fix (documented in `project/remaining_tasks.md`, Task 1) was to add a 9-deep pixel FIFO on the AXI write path. Instead of requiring the host to interleave PIXEL_IN and CTRL writes (2 AXI transactions per pixel tap = 18 transactions for 9 pixels), the host would stream all 9 pixels in 9 consecutive PIXEL_IN writes with a single trailing START strobe. The FIFO would auto-generate `valid_in` pulses at one per clock, reducing per-patch overhead from 142 AXI cycles to approximately 10 cycles and raising throughput toward ~240 MFLOPs/s.

The RTL design for this modification was sketched but not implemented because adding a FIFO to `interface.sv` requires changing the `valid_out` generation from a CTRL-write event to a FIFO-read event, which propagates changes into `top.sv` and `tb_top.sv`. With the M4 deadline five days away at the time of implementation, introducing a new FSM structure and re-verifying the testbench was judged too risky. The correct approach for a future revision is to implement and verify the FIFO modification in isolation before integrating it with the top-level.

9.2 8-PE expansion and `weight_addr` widening (Task 2 — not completed).

CF07 (`project/codefest/cf07/`) identified that `compute_core.sv` uses a 2-bit `weight_addr` port, which silently truncates when `NUM_PE > 4`. Expanding to 8 PEs doubles throughput per clock cycle but requires widening `weight_addr` from `[1:0]` to `[2:0]` in `compute_core.sv` and in `top.sv` (`w_addr_ctr` and `weight_addr_lat` registers). A companion change to `tb_top.sv` is needed to load 8 weights. This was not completed for the same reason as Task 1: the risk of introducing an undetected truncation bug across three files outweighed the throughput benefit when the bottleneck is AXI overhead, not PE count.

9.3 OpenLane 2 detailed routing interruption (Task 3 — partially completed).

The OpenLane 2 flow (`project/m3/synth/runs/RUN_2026-05-20_22-44-53/`) completed all 42 steps through TritonRoute detailed routing launch (step 43). However, TritonRoute was interrupted mid-run when the WSL2 host process was killed by the Windows memory manager — the TritonRoute run consumed approximately 6 GB of RAM, exceeding the WSL2 virtual memory limit on the development machine. The detailed routing log is empty; no `state_out.json` was produced for step 43.

As a result, the M4 synthesis reports use post-global-routing numbers from step 42 (STAMidPNR-3) rather than post-detailed-routing numbers. The practical impact is minimal for timing: post-global-routing STA is typically within 5–10% of post-route STA for designs with large positive slack (+4.062 ns here). Power and area numbers are unaffected by routing completion. If OpenLane 2 were re-run on a machine with 16 GB dedicated RAM or in a cloud container, the flow would complete and a GDSII file would be generated.

9.4 M3 STATUS register bug (discovered and fixed).

In the M2 `axi4lite_slave`, the STATUS register was incorrectly cleared every cycle when `reg_ctrl[0]` was set, because the stored CTRL register is not self-clearing. This caused STATUS to pulse for only one cycle after `done_in`, making it impossible for a slow host to poll STATUS reliably. M3 fixed this by making STATUS sticky on `done_in` and only resetting on `rst` (see `project/m3/rtl/interface.sv`, line 113). The M3/M4 testbench confirmed the fix: polling DONE returns 1 on the first poll in all three invocations.

9.5 Lessons learned.

- Run OpenLane 2 on a dedicated Linux environment (not WSL2) from the start. The virtual memory limits of WSL2 are hidden until TritonRoute allocates its working set.
- Implement the streaming FIFO before expanding PE count. The FIFO eliminates the dominant bottleneck; more PEs at 5.96% utilization provide near-zero benefit until bus overhead is resolved.
- Add `$time` markers to testbenches from the beginning. The M3 testbench did not print timing, which led to benchmark numbers that could not be traced back to the log. The M4 testbench was corrected to include explicit TIMING SUMMARY output with cycle counts derived directly from `$time` readings.

- The $8.07\times$ speedup over a Python loop is real but context-dependent: NumPy vectorized convolution achieves 1,527 MFLOPs/s, meaning the hardware at 47.7 MFLOPs/s is slower than optimized NumPy. This confirms the FIFO is the critical next design step before claiming a production-relevant speedup.

*All numerical data in this report is traceable to `project/m4/bench/benchmark_data.csv`. Timing numbers (cycles/patch, ns/patch) are directly printed in `project/m4/sim/final_run.log` under **TIMING SUMMARY**. RTL source files are in `project/m4/rtl/`. Synthesis reports are in `project/m4/synth/`. Figures: Figure 1 (`block_diagram.png`), Figure 2 (`dataflow_diagram.png`), Figure 3 (`final_waveform.png`), Figure 4 (`roofline_final.png`) — all in `project/m4/report/figures/`.*